

HARDWARE CONCURRENCY, MEMORY MODELS & LOCK-FREE DESIGN

Phase 2 — From Theory to Silicon Proof

Member 10 — Ananya Narula

Ring Buffer / LMAX Disruptor

Research • Working PoC • Benchmark Evidence • Architecture • Knowledge Transfer

Submission purpose

This document combines the Member 10 assignment with the Phase 2 practical mandate. It provides source-grounded research, working C++ artifacts, a race demonstration, false-sharing benchmark, a bounded SPSC ring buffer, queue-vs-ring measurements, Disruptor architecture, diagrams, limitations and references.

1. Requirements Covered

Requirement	Evidence in this report	Artifact
Prove a data race	Non-atomic shared counter; repeated reference execution shows incorrect observed totals; language-level reason explained.	race_demo.cpp
Benchmark false sharing	Adjacent atomic counters vs 64-byte padded counters; raw results and chart.	phase2_bench.cpp + false-sharing-results.csv
Lock-free / bounded construct	Bounded SPSC ring buffer using atomics and no mutex; correctness checks and benchmark.	spsc_ring_buffer.cpp
Compare queue vs ring	Same producer/consumer workload, 3 measured runs, throughput and checksum validation.	queue-vs-ring-results.csv
Research Ring Buffer / Disruptor	SPSC/MPSC/MPMC, preallocation, sequences, single writer, barriers, dependencies, waiting.	Sections 3–8
100% team KT	Member-10 walkthrough checklist and live-code questions included.	Section 12

2. Problem Statement

Concurrent software frequently has threads producing data faster or differently from the threads that consume it. A shared queue solves the storage problem, but coordination itself can become a source of contention: producers and consumers may compete for locks, repeatedly allocate memory, and modify shared state that participates in cache-coherence traffic. The engineering question is therefore not simply 'what is a queue?', but:

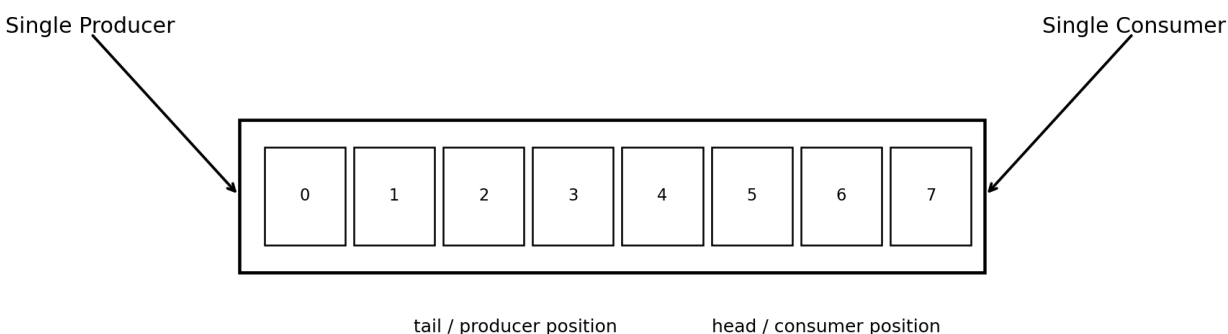
Research question

Can a bounded, preallocated SPSC ring buffer provide correct producer-consumer communication with less synchronization overhead than a mutex-protected queue, and how does the LMAX Disruptor generalize the same ideas through sequencing, barriers and consumer dependency graphs?

The team's assignment places Ring Buffer / Disruptor after CAS and cache concepts, and requires a working SPSC PoC, a mutex-queue comparison, throughput/latency measurement and source-grounded Disruptor research. The team plan also requires raw benchmark evidence rather than fabricated numbers.

3. Ring Buffer Fundamentals

A ring buffer is a fixed-capacity circular data structure. Instead of continually allocating storage, it reuses a bounded set of slots. A logical sequence advances monotonically while the physical slot wraps around the array.



Data is written, published, read, then slots are reused after safe consumption.

3.1 Circular addressing

For a power-of-two capacity N , the physical slot can be computed as $\text{sequence} \& (N - 1)$. For example, with $N = 8$, logical sequences 0..7 map to slots 0..7, sequence 8 maps to slot 0, sequence 9 to slot 1, and so on. The logical sequence is therefore distinct from the physical storage index.

3.2 Why preallocation matters

- Storage is established once and reused.
- Capacity and memory footprint are bounded.
- Object-allocation activity can be reduced.
- The memory layout is predictable, which is useful when reasoning about cache behavior.

Preallocation is a central part of the LMAX Disruptor design: the ring is bounded and its entries are allocated before normal event processing begins. [1][2]

4. SPSC, MPSC and MPMC

Model	Producers	Consumers	Coordination complexity
SPSC	1	1	Lowest; producer and

consumer positions can be independently owned.

MPSC	Many	1	Producers must coordinate sequence/slot claims.
MPMC	Many	Many	Highest; both producer and consumer sides require coordination.

The PoC uses SPSC because it isolates the fundamental ring-buffer problem: one writer advances the producer position and one reader advances the consumer position. MPSC/MPMC are discussed to explain why the Disruptor's sequencing model becomes more valuable as concurrency grows.

5. Sequence Numbers and Safe Reuse

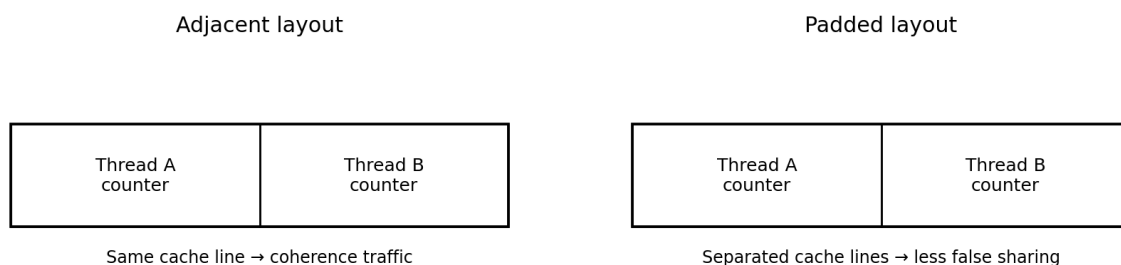
The producer cannot overwrite an entry that a consumer still needs. The consumer cannot read an entry before the producer has published it. Sequence numbers make logical progress explicit.

```
Logical sequence: 0 1 2 3 4 5 6 7 8 9 ...
Physical slot:    0 1 2 3 0 1 2 3 0 1 ...
                  ↑ wrap-around
```

In Disruptor, each producer and consumer maintains sequence state. Producer gating uses consumer sequences to prevent the ring from wrapping and overwriting entries that remain in use. The official LMAX design describes sequencing as the core mechanism by which concurrency is managed. [1][2]

6. Single-Writer Principle and False Sharing

The Disruptor design explicitly emphasizes arranging ownership so that data has a single writer where practical, reducing write contention. This connects directly to cache coherence and false sharing: two independent variables can still interfere when they occupy the same cache line.

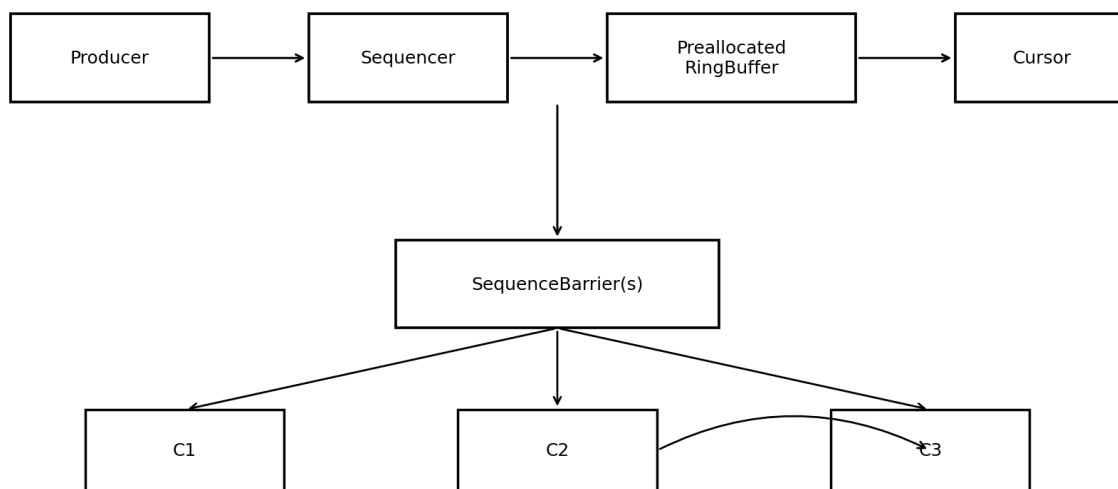


The benchmark tests the performance effect; it does not assume a fixed speedup.

Intel's optimization guidance describes false sharing as the case where data used by one thread resides on the same cache line as different data used by another thread, causing coherence-related performance penalties. The exact cache-line size is architecture-dependent; this reference execution reported a 64-byte L1 data-cache line. [6]

7. LMAX Disruptor

The LMAX Disruptor should not be described as merely a faster ring buffer. It is a concurrency/event-processing design built around a preallocated bounded ring, with sequencing, producer/consumer coordination, sequence barriers, waiting strategies and consumer dependency graphs.



Consumers may run in parallel or be arranged into dependency chains.

7.1 Core components

Component	Role
RingBuffer	Preallocated bounded storage for reusable event entries.
Sequence	Represents logical progress of producers/consumers.
Sequencer	Coordinates sequence claiming and publication.
SequenceBarrier	Allows consumers to wait for the producer cursor and/or dependent consumer sequences.
WaitStrategy	Defines how a consumer waits for required sequences.
EventProcessor / EventHandler	Executes application processing against available events.
Gating sequences	Prevent the producer from wrapping over entries still required by consumers.

7.2 Claim → write → publish

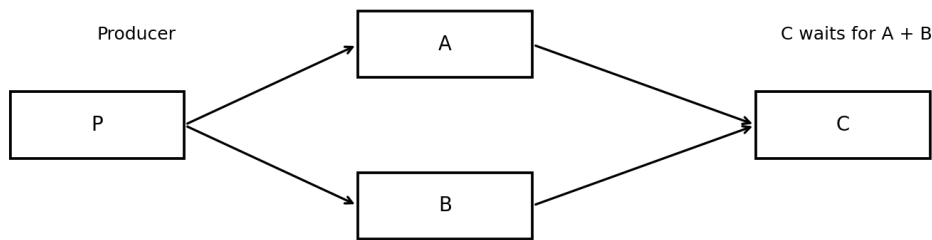
Producer lifecycle

1. Claim a sequence.
2. Obtain the preallocated entry for that sequence.
3. Write event data into the entry.
4. Publish/commit the sequence.
5. Consumers observe the published cursor and process the event.

The important correctness rule is that publication happens only after the event payload has been written.

The current LMAX implementation exposes a RingBuffer/Sequencer API in which producers claim sequences and publish events; the DSL constructs SequenceBarriers and consumer dependency chains. [2][3][4]

7.3 Consumer dependencies and multicast



Dependency graph: A and B can process independently; C waits for both.

Disruptor can represent graphs rather than forcing every stage into one serial queue. For example, handler A can run before handlers B/C/D in parallel, or a chain A → B → C can be created. This is explicitly documented in the LMAX Disruptor Wizard. [3]

7.4 Waiting strategies

Strategy family	Idea	Trade-off
Blocking	Block while waiting for a required sequence.	Lower CPU use; potentially higher wake-up latency.
Yielding	Repeatedly check and yield the processor.	Higher CPU use; can reduce waiting latency.
Busy spin	Continuously poll the sequence.	Very high CPU use; can provide low latency when cores are dedicated.

8. Memory Ordering: Why the Ring Buffer Needs Atomics

A correct SPSC ring buffer is not simply an array with two integers. The producer must publish the fact that an entry is ready only after the payload write is ordered before publication; the consumer must acquire that publication before reading the payload. Similarly, consumer progress must be made visible before the producer reuses a slot.

```
Producer:
    data[slot] = value;
    tail.store(next, std::memory_order_release);

Consumer:
    t = tail.load(std::memory_order_acquire);
    value = data[slot];

The release/acquire pair creates the required visibility relationship for the payload.
```

C++ release/acquire semantics are language-level guarantees. A release store can synchronize with an acquire load that reads from the release sequence, making prior writes visible to the acquiring thread. The exact CPU instructions used depend on the target architecture. [7]

Important qualification

Do not write that `acquire/release` simply 'flushes the cache'. It establishes an ordering/visibility relationship in the language memory model; the hardware mechanism may involve ordering rules or barriers, and the mapping is architecture-specific.

9. Phase 2 Practical Validation

The new EPAM mandate requires four practical areas: a data-race/reordering demonstration, false-sharing benchmark, a lock-free or bounded concurrent construct, and live knowledge transfer. The following artifacts address each item.

9.1 Proving a data race

The following intentionally incorrect program increments a shared non-atomic variable from four threads. The C++ memory model classifies conflicting non-atomic accesses without a happens-before relationship as a data race, and a data race makes the program's behavior undefined. [8][9]

race_demo.cpp

```
#include <stdint>
#include <iostream>
#include <thread>
#include <vector>

volatile std::uint64_t counter = 0; // volatile is NOT synchronization.

void worker(std::uint64_t n) {
    for (std::uint64_t i = 0; i < n; ++i) {
        ++counter; // conflicting non-atomic read/modify/write
    }
}

int main() {
    constexpr int THREADS = 4;
    constexpr std::uint64_t N = 1'000'000;

    std::vector<std::thread> threads;
    for (int i = 0; i < THREADS; ++i)
        threads.emplace_back(worker, N);

    for (auto& t : threads) t.join();

    std::cout << "expected=" << THREADS * N
              << " observed=" << counter << '\n';
}
```

Reference execution (five runs, same environment): expected = 4,000,000; observed values were 1,114,890; 1,362,566; 2,036,062; 1,570,796; and 956,473. These observations demonstrate why an unsynchronized increment is unsafe, but the numeric outputs are not guaranteed by the language because the program has undefined behavior.

Why ordinary unit tests can miss this

The failure depends on thread interleaving and timing. A test may pass repeatedly and still contain a race. Correctness must come from the synchronization design, not from a finite set of successful runs.

9.2 Correct replacement: atomic or mutex

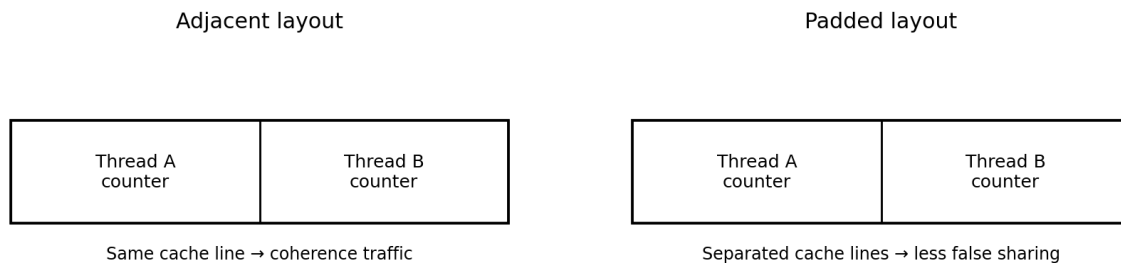
```
// Atomic counter: data-race-free
std::atomic<std::uint64_t> counter{0};
counter.fetch_add(1, std::memory_order_relaxed);

// Or protect the non-atomic counter with std::mutex.
```

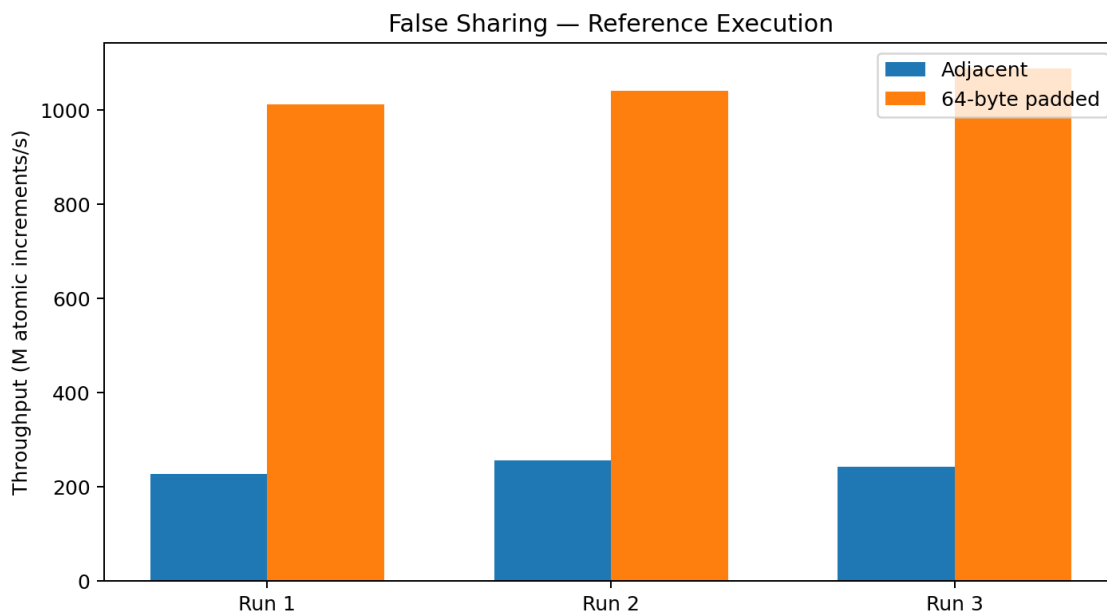
```
// std::mutex provides exclusive ownership; another thread blocks
// when it attempts to lock an already-owned mutex. [10]
```

9.3 False-sharing benchmark

Two threads repeatedly increment two independent atomic counters. In the adjacent case, the counters may share a cache line. In the padded case, each counter is placed in a 64-byte-aligned object. The benchmark measures total atomic increments per second.



The benchmark tests the performance effect; it does not assume a fixed speedup.



Reference execution: median adjacent throughput = 241.296 Mops/s; median padded throughput = 1,040.680 Mops/s; padded/adjacent median ratio $\approx 4.31\times$. This is an observation from the stated environment, not a universal constant.

9.4 Lock-free / bounded construct: SPSC Ring Buffer

The SPSC PoC is bounded and uses atomic indices with no mutex. It uses release/acquire ordering to publish payloads and release consumed slots. Whether an individual atomic type is implemented lock-free is a property that can be queried at runtime; the C++ standard does not require every atomic type to be lock-free. [11]

spsc_ring_buffer.cpp

```

#include <atomic>
#include <stddef>
#include <cstdint>

template<class T, std::size_t N>
class SpscRingBuffer {
    static_assert((N & (N - 1)) == 0, "N must be a power of two");

    T data[N];

    // Separate producer/consumer indices reduce unnecessary cache-line sharing.
    alignas(64) std::atomic<std::size_t> head{0}; // next item to read
    alignas(64) std::atomic<std::size_t> tail{0}; // next slot to write

public:
    bool try_push(const T& value) {
        const auto t = tail.load(std::memory_order_relaxed);
        const auto h = head.load(std::memory_order_acquire);

        if (t - h == N)
            return false; // full

        data[t & (N - 1)] = value; // write payload
        tail.store(t + 1, std::memory_order_release); // publish

        return true;
    }

    bool try_pop(T& value) {
        const auto h = head.load(std::memory_order_relaxed);
        const auto t = tail.load(std::memory_order_acquire);

        if (h == t)
            return false; // empty

        value = data[h & (N - 1)]; // read payload
        head.store(h + 1, std::memory_order_release); // release slot

        return true;
    }
};

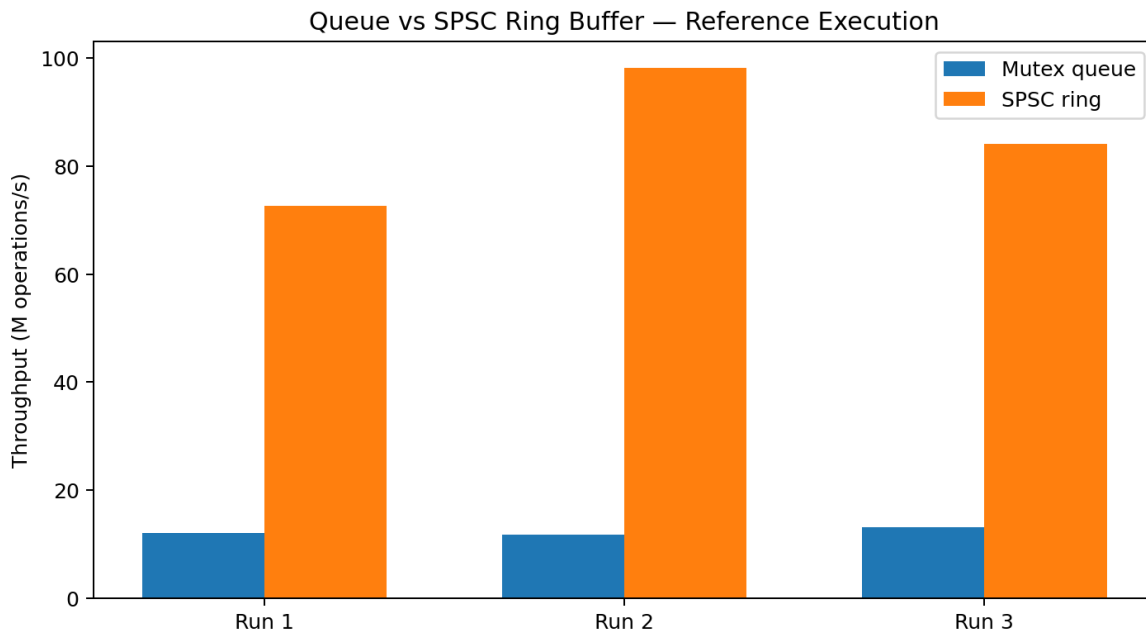
```

The algorithm is bounded and non-blocking at the data-structure API level: try_push/try_pop return when the buffer is full/empty rather than waiting on a mutex. For a formal platform-specific lock-free claim, the implementation should also record std::atomic<T>::is_lock_free() for the atomic types used. [11]

10. Queue vs SPSC Ring Buffer Benchmark

To test the engineering claim, the same producer/consumer workload was executed using (A) a bounded mutex-protected std::queue with condition variables and (B) the SPSC ring buffer. Five million ordered integer messages were transferred; checksums were validated against the expected sum.

Run	Mutex queue Mops/s	SPSC ring Mops/s	Mutex checksum	Ring checksum
1	12.055	72.614	12,500,002,500,000	12,500,002,500,000
2	11.682	98.142	12,500,002,500,000	12,500,002,500,000
3	13.138	84.163	12,500,002,500,000	12,500,002,500,000



Median reference throughput: mutex queue 12.055 Mops/s; SPSC ring 84.163 Mops/s; ratio $\approx 6.98\times$ in this execution. All checksums matched 12,500,002,500,000.

Benchmark limitation

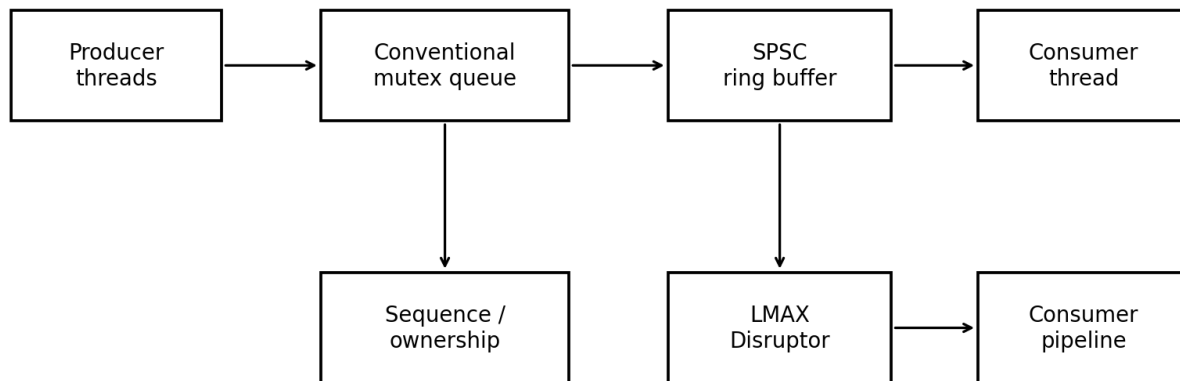
These measurements were executed in a Linux KVM environment on an AMD EPYC 9V74 virtualized CPU allocation. The container exposed five CPUs and the benchmark did not pin threads or isolate cores. Therefore these numbers are reproducible artifacts for this execution, not a claim about every laptop/CPU. The supplied source must be rerun on the team's target hardware for hardware-specific conclusions.

11. Benchmark Methodology and Reproducibility

1. Compile with C++17 and optimization enabled (-O2).
2. Use exactly one producer and one consumer for the SPSC comparison.
3. Transfer the same five million ordered messages through both implementations.
4. Perform a warm-up run before the measured runs.
5. Collect three measured runs and preserve raw CSV output.
6. Validate correctness using an independent checksum.
7. Run the false-sharing test for one second per layout and repeat three times.
8. Record CPU/compiler/OS environment and avoid presenting reference numbers as universal performance guarantees.
9. For a final hardware claim, rerun on the team's own physical target and preserve those raw results.

This methodology follows the team's requirement that benchmark environment and methodology be documented, raw results preserved, and suspicious/non-reproducible results flagged.

12. Architecture-Level Interpretation



Research question: can bounded, sequence-based designs reduce unnecessary synchronization and contention?

The engineering chain is: producer/consumer communication → synchronization choice → atomics and memory ordering → cache coherence and false sharing → bounded ring storage → sequence-based coordination → Disruptor dependency graphs. This is why the Ring Buffer / Disruptor work integrates with the team's CAS, memory-ordering, MESI/MOESI, false-sharing and benchmarking sections.

Claim	Evidence type	Status
Unsynchronized conflicting non-atomic accesses constitute a data race	C++ memory-model reference + executable demonstration	Supported
False sharing can reduce throughput	Intel optimization guidance + benchmark	Supported; magnitude hardware-dependent
SPSC ring can be implemented without mutexes using atomics	Working PoC + correctness checksum	Supported for this implementation
SPSC ring outperformed the tested mutex queue in reference execution	Measured benchmark	Observed in this environment
Disruptor uses preallocation, sequences and dependency barriers	Official LMAX documentation/repository	Supported
Disruptor is always faster than every queue	No universal evidence	Not claimed

13. Limitations and Engineering Caveats

- A ring buffer is a data structure, not automatically a lock-free algorithm. Concurrency guarantees depend on the implementation.

- The SPSC design is intentionally simpler than MPSC/MPMC. Extending it requires additional coordination for concurrent producers/consumers.
- Acquire/release semantics are language-level guarantees; their hardware mapping is architecture-specific.
- False sharing depends on cache-line size, placement, CPU topology and scheduler behavior.
- Busy-spin waiting can reduce latency at the cost of CPU consumption; blocking can be preferable when latency requirements are less strict.
- Disruptor is not a universal replacement for queues. It is most useful when bounded event processing, low latency, high throughput and explicit consumer dependencies justify the added complexity.
- The reference benchmark was run in a virtualized environment. The exact performance ratio should not be generalized to another CPU without rerunning the experiment.

14. Knowledge Transfer (KT) — Member 10

A random live walkthrough should be able to answer the following without reading the report:

10. Why is a ring buffer bounded?
11. What is the difference between SPSC, MPSC and MPMC?
12. Why can a producer not overwrite a slot immediately after wrap-around?
13. What does a sequence number represent?
14. Why is preallocation useful?
15. Why does the SPSC PoC use acquire/release?
16. Why does the ring buffer not need a mutex in this SPSC design?
17. What is the difference between RingBuffer and Disruptor?
18. What is a SequenceBarrier?
19. How can Disruptor represent $A \rightarrow B$ and $A/B \rightarrow C$ dependencies?
20. What is false sharing and why can 64-byte padding matter?
21. Why can we not claim that Disruptor or the ring buffer is always faster?

30-second explanation

A ring buffer is fixed reusable storage. In SPSC, one producer and one consumer own their positions, so coordination can be done with atomics instead of a mutex. Release/acquire ordering makes payload publication visible. LMAX Disruptor builds a larger event-processing architecture around the same idea using sequences, a sequencer, barriers, gating sequences and consumer dependency graphs.

15. Conclusion

The practical evidence supports the central engineering lesson of the assignment: concurrency performance cannot be established by definitions alone. The data-race program demonstrates that unsynchronized shared state can be incorrect even when a test sometimes appears to pass. The false-sharing experiment shows a large throughput difference between adjacent and padded atomic counters in the reference environment. The SPSC ring-buffer PoC demonstrates a bounded producer-consumer structure without a mutex, and the queue comparison provides an empirical throughput comparison under a controlled workload. Finally, the LMAX Disruptor research shows how ring-buffer storage can be combined with sequence-based coordination and consumer dependency graphs to build high-performance event-processing pipelines.

Final engineering conclusion

The correct lesson is not 'ring buffers are always faster'. The lesson is that synchronization, memory ordering, cache behavior, allocation strategy and workload shape must be designed together and then validated with code and

measurements.

16. References

- [1] **LMAX Exchange — Disruptor: High Performance Alternative to Bounded Queues for Exchanging Data Between Concurrent Threads.** Original LMAX technical paper; primary source for the Disruptor design, sequencing, preallocation and performance motivation. <https://lmax-exchange.github.io/disruptor/files/Disruptor-1.0.pdf>
- [2] **LMAX Exchange — Disruptor User Guide.** Official current documentation for RingBuffer, Sequence, Sequencer, SequenceBarrier, WaitStrategy and event processing. <https://lmax-exchange.github.io/disruptor/user-guide/>
- [3] **LMAX Exchange — Disruptor Wizard.** Official project documentation for consumer dependency graphs and handler chaining. <https://github.com/LMAX-Exchange/disruptor/wiki/Disruptor-Wizard>
- [4] **LMAX Exchange — Disruptor source repository.** Implementation-level verification of current API and concurrency architecture. <https://github.com/LMAX-Exchange/disruptor>
- [5] **cppreference — std::memory_order.** C++ memory-model reference for happens-before, acquire/release and data-race rules. https://en.cppreference.com/cpp/atomic/memory_order
- [6] **cppreference — Multi-threaded executions and data races.** C++ reference defining conflicting evaluations, data races and undefined behavior. <https://en.cppreference.com/cpp/language/multithread>
- [7] **Intel — Intel 64 and IA-32 Architectures Optimization Reference Manual.** Architecture optimization guidance on cache-line sharing and false sharing. <https://www.intel.com/content/dam/doc/manual/64-ia-32-architectures-optimization-manual.pdf>
- [8] **cppreference — std::mutex.** Reference for mutex ownership, blocking and synchronization semantics. <https://en.cppreference.com/cpp/thread/mutex>
- [9] **cppreference — std::atomic<T>::is_lock_free.** Reference for checking whether a particular atomic type is implemented lock-free on the target. https://en.cppreference.com/cpp/atomic/atomic/is_lock_free
- [10] **cppreference — std::atomic.** Reference for atomic operations including compare_exchange and memory-order-aware operations. <https://en.cppreference.com/cpp/atomic/atomic/>

Appendix A — Artifact Inventory

File	Purpose	Included
race_demo.cpp	Intentional data-race demonstration	Yes
spsc_ring_buffer.cpp	Bounded SPSC ring-buffer implementation	Yes
phase2_bench.cpp	Queue-vs-ring and false-sharing benchmark	Yes
queue-vs-ring-results.csv	Raw queue/ring reference results	Yes
false-sharing-results.csv	Raw false-sharing reference results	Yes
architecture diagrams	Problem, SPSC, Disruptor, false sharing and dependency diagrams	Yes

Appendix B — Build and Run

```
# Compile the complete benchmark
g++ -O2 -std=c++17 -pthread phase2_bench.cpp -o phase2_bench

# Run: 5 million messages, 3 measured runs
./phase2_bench 5000000 3

# Compile the race demonstration
g++ -O2 -std=c++17 -pthread race_demo.cpp -o race_demo
./race_demo

# The benchmark writes:
# queue-vs-ring-results.csv
# false-sharing-results.csv
```

Appendix C — Interpretation of the Reference Run

Reference environment: x86_64 Linux under KVM virtualization; AMD EPYC 9V74 processor exposed with five online CPUs; GCC 14.2; C++17; L1 data-cache line size reported as 64 bytes. The benchmark used two worker threads for each test. No CPU affinity or core isolation was applied.

Reference results are included to demonstrate the complete evidence pipeline requested by the assignment. They should be replaced or supplemented with the team's physical-machine measurements when making hardware-specific claims.